

Theory Assignment 5

Aayush Kumar - CO21BTECH11002

1 Q1 (6.3)

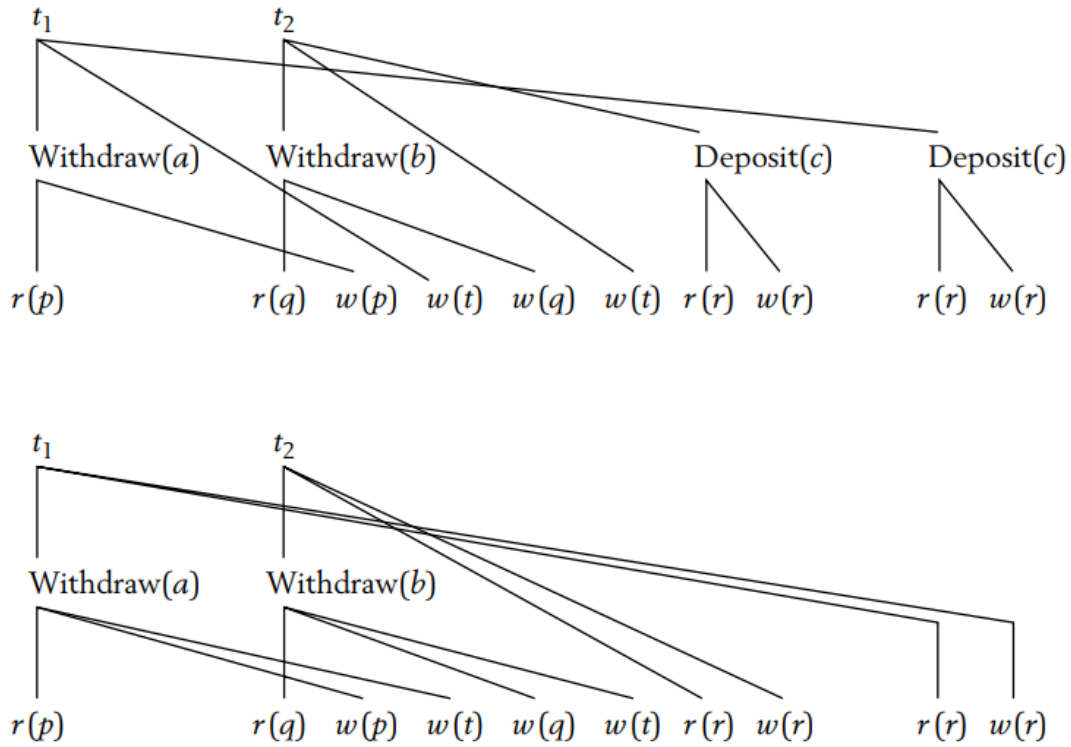


Figure 1: Fig 6.2 from the book

In the first schedule, at level L_0 , we can isolate $Withdraw_2(b)$ by shifting $r_2(q)$ to the right, placing it after $w_1(t)$. All operations at level L_1 are already isolated, allowing us to prune the operations at level L_0 . We then insert $Deposit_1(c)$ between $Withdraw_1(a)$ and $Withdraw_2(b)$. This results in the serialization $t_1 <_s t_2$.

In the second schedule, we can isolate both $Withdraw_1(a)$ and $Withdraw_2(b)$ by shifting $r_2(q)$ to the right, placing it after $w_1(t)$. However, we do not prune the corresponding subtrees at this

stage. Next, we shift $Withdraw_1(a)$ so that its entire subtree is positioned between $w_2(r)$ and $r_1(r)$. This reordering is permissible, as it does not introduce any page-level conflicts. As a result, we isolate t_1 and t_2 with the serialization order $t_2 <_s t_1$.

Hence, both the schedules are tree reducible.

2 Q2 (6.6)

We consider a bounded counter that supports three operations: **increment**, **decrement**, and **getValue**, abbreviated as **inc**, **dec**, and **gv**, respectively. Each operation returns a value: “OK” or “No” for **inc** and **dec**, and the counter value for **gv**. We examine which pairs of these operations are **return-value commutative**, i.e., their effects and return values are independent of the order in which they are applied.

	inc↑ok	inc↑no	dec↑ok	dec↑no	gv↑res
inc↑ok	+	−	−	+	−
inc↑no	+	+	−	+	+
dec↑ok	−	+	+	−	−
dec↑no	−	+	+	+	+
gv↑res	−	+	−	+	+

Legend: “+” indicates the operations are commutative; “−” indicates they are not.

To enhance concurrency further, we can take into account explicit operation parameters and initial counter state x_0 . Pairs of operations that are usually non-commutative might become commutative if their effects are constrained by bounds:

- If $x_0 + \Delta_2 \leq \text{upper_bound}$, then $\text{dec}_1(x, \Delta_1) \uparrow \text{ok}$ and $\text{inc}_2(x, \Delta_2) \uparrow \text{ok}$ are commutative.
- If $x_0 + \Delta_2 \geq \text{upper_bound}$, then $\text{inc}_1(x, \Delta_1) \uparrow \text{ok}$ and $\text{inc}_2(x, \Delta_2) \uparrow \text{no}$ are commutative.
- If $x_0 - \Delta_2 \geq \text{lower_bound}$, then $\text{inc}_1(x, \Delta_1) \uparrow \text{ok}$ and $\text{dec}_2(x, \Delta_2) \uparrow \text{ok}$ are commutative.
- If $x_0 - \Delta_2 \leq \text{lower_bound}$, then $\text{dec}_1(x, \Delta_1) \uparrow \text{ok}$ and $\text{dec}_2(x, \Delta_2) \uparrow \text{no}$ are commutative.
- If $x_0 - \Delta_2 + \Delta_1 \leq \text{upper_bound}$, then $\text{inc}_1(x, \Delta_1) \uparrow \text{no}$ and $\text{dec}_2(x, \Delta_2) \uparrow \text{ok}$ are commutative.
- If $x_0 + \Delta_2 - \Delta_1 \leq \text{lower_bound}$, then $\text{dec}_1(x, \Delta_1) \uparrow \text{no}$ and $\text{inc}_2(x, \Delta_2) \uparrow \text{ok}$ are commutative.

Thus, by analyzing parameter constraints, more fine-grained commutativity and concurrency can be achieved.

3 Q3 (7.1)

First, consider layered schedules. When a transaction initiates an operation at level L_i , it first acquires the corresponding lock at that level. At this point, the transaction does not hold any locks at lower levels. Subsequently, the transaction initiates sub-operations at level L_{i-1} . Once the operations at level L_{i-1} complete, the locks at that level are released, while the lock at level L_i is retained until its operation finishes. Based on this behavior, and using induction, we can conclude

that if a transaction is blocked at level L_i , it cannot hold locks at any lower levels. Therefore, it cannot block other transactions at those lower levels.

Now, suppose there is a “global” deadlock such that transaction t_1 waits for t_2 at L_{i_1} , t_2 waits for t_3 at L_{i_2} , and so on, until t_n waits for t_1 at L_{i_n} , with $n \geq 2$. In this scenario, t_2 must be holding a lock at L_{i_1} while requesting a lock at L_{i_2} , implying $i_1 \geq i_2$. By induction, we derive $i_1 \geq i_2 \geq \dots \geq i_n \geq i_1$, which can only hold if all levels i_j are equal. Hence, all locks involved in such a deadlock reside at the same level. Therefore, cross-level deadlock detection is unnecessary.

If we allow non-layered schedules, global deadlocks must be considered. Assume a two-level architecture involving a record level and a page level. Transaction t_1 first reads a page p (acquiring a read lock), and then attempts a record-level operation $modify(x)$. Transaction t_2 also performs a record-level operation $modify(x)$, and subsequently attempts to write to the page p . However, $w(p)$ is blocked since the page is already locked for reading by t_1 . At the same time, t_1 ’s modify operation is blocked due to t_2 ’s action. This causes a deadlock: t_1 is blocked at the record level while still holding the page-level lock, which in turn prevents t_2 from making progress.